



Argo Rollouts

Déploiement progressif & rollback automatique sur Kubernetes

Plan de la présentation

1

La technologie

Le problème, la définition, le fonctionnement

2

Cas d'usage professionnel

Où, comment, avantages & limites, écosystème

3

Mise en œuvre technique

Architecture, choix, démonstration live

4

Retour d'expérience

Bilan, avis personnel, améliorations

Le problème : le Deployment classique

Un **Deployment** fait un **rolling update** : il remplace les pods un à un... sans jamais vérifier si la nouvelle version fonctionne.

Les risques

- × Une v2 buggée part vers 100 % des utilisateurs
- × Détection tardive : alertes, support, churn
- × Rollback manuel, sous stress, en plein incident
- × Aucune mesure objective avant de généraliser

Le besoin

- ▶ Exposer la v2 à une fraction du trafic d'abord
- ▶ Mesurer sa santé en conditions réelles
- ▶ Promouvoir seulement si les métriques sont bonnes
- ▶ Revenir en arrière automatiquement sinon

DÉFINITION

Qu'est-ce qu'Argo Rollouts ?

Un **contrôleur Kubernetes** (projet CNCF, famille Argo) qui remplace le Deployment par une ressource **Rollout** offrant la progressive delivery.

Canary

- ▶ % croissant du trafic vers la v2
- ▶ 20% → 40% → 60% → 100%

Blue-Green

- ▶ Deux environnements isolés
- ▶ Bascule instantanée

Analyse auto

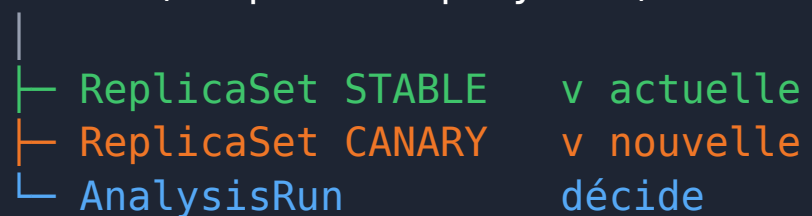
- ▶ Décision pilotée par métriques
- ▶ Promotion ou rollback

→ Objectif : une mise en production **graduelle, mesurée et réversible automatiquement.**

Fonctionnement global

ressources gérées

Rollout (remplace Deployment)



exemple de stratégie

steps:

- setWeight: 20
- pause: { duration: 30s }
- analysis: { success-rate }

À chaque palier, le contrôleur

- ▶ Ajuste la proportion de trafic/pods canary
- ▶ Marque une pause (minutée ou manuelle)
- ▶ Lance une analyse de la santé du canary
- ✓ **Promeut si OK**
- × **Rollback automatique si KO**

Cas d'usage en entreprise & écosystème

Utilisation en production

- ▶ Réduire le risque : tester la v2 sur 5-10 % du trafic
- ▶ Déployer plusieurs fois/jour sans maintenance
- ▶ SLO-driven : promotion liée aux métriques
 - latence, taux d'erreur, succès (Prometheus, Datadog...)
- ▶ Couplé au GitOps (ArgoCD / Flux)

Écosystème

- ▶ Projet CNCF (Argoproj), combo standard avec ArgoCD
- ▶ Voisins : Flagger (Flux), Spinnaker
- ▶ Analyse native : Prometheus, Datadog, New Relic,
 - CloudWatch, Graphite, Job / Web custom
- ▶ Contextes : microservices, API critiques, e-commerce, SaaS

Avantages & limites

Avantages

- ✓ **Rollback automatique → moins d'incidents**
- ✓ **Pas de downtime, exposition contrôlée**
- ✓ **Décision objective (métriques), pas « au feeling »**
- ✓ **Intégration native : ArgoCD, Prometheus, Istio**

Limites

- × **Complexité accrue (CRD, analyse, parfois mesh)**
- × **Trafic fin = Istio / NGINX / SMI requis**
- × **Nécessite de bonnes métriques (sinon fausse confiance)**
- × **Courbe d'apprentissage, observabilité indispensable**

Architecture de la démo

flux GitOps de bout en bout

git push → GitHub Actions (lint + validation des manifests)

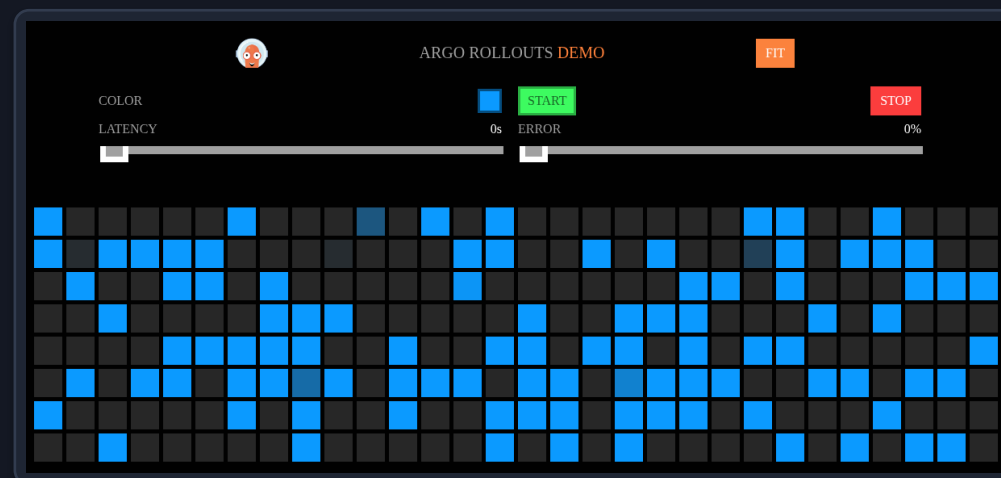
└─> ArgoCD (pull) → Rollout → Contrôleur Argo Rollouts

└─ ReplicaSet stable / canary

└─ AnalysisRun (Job de sonde HTTP)

Environnement & astuce

- ▶ Cluster kind + ArgoCD + Argo Rollouts v1.7.2
- ▶ App rollouts-demo : 1 couleur = 1 version
- ✗ **Le tag bad-red renvoie des HTTP 500**
 - version « buggée » déterministe pour le rollback



Capture réelle de l'UI de l'app de démo

Choix techniques clés

4 décisions structurantes

- ▶ Canary sans service mesh → split par nombre de pods (simple, portable)
- ▶ Services stable + canary : le contrôleur y injecte le pod-template-hash
 - la sonde cible ainsi précisément les pods canary
- ▶ Probes TCP (et non HTTP /color) : les pods bad-red restent Ready malgré les 500
 - on démontre un rollback piloté par l'ANALYSE, pas par la santé
- ▶ AnalysisTemplate de type Job : sonde 30 requêtes, exige ≥ 95 % de succès
 - autonome ; variante Prometheus fournie pour la production

Démo 1 — Canary sain

Scénario

- ▶ Déploiement de la version green
- ▶ Canary 20% → 40%
- ▶ AnalysisRun sonde le canary
- ✓ **30/30 = 100 % (seuil 95%)**
- ✓ **Promotion auto à 100 %**

kubectl argo rollouts get rollout - canary en cours

```
Name:          rollouts-demo
Namespace:     demo
Status:        [ ] Paused
Message:       CanaryPauseStep
Strategy:      Canary
  Step:        3/9
  SetWeight:   40
  ActualWeight: 40
Images:        argoproj/rollouts-demo:blue (canary)
               argoproj/rollouts-demo:green (stable)

Replicas:
  Desired:     5
  Current:     5
  Updated:     2
  Ready:       5
  Available:   5
```

NAME	KIND	STATUS	AGE	INFO
[] rollouts-demo	Rollout	[] Paused	24m	
# revision:5				
└─ [] rollouts-demo-774b46cfdc	ReplicaSet	✓ Healthy	24m	canary
└─└─ [] rollouts-demo-774b46cfdc-6z47x	Pod	✓ Running	50s	ready:1/1
└─└─ [] rollouts-demo-774b46cfdc-x99l2	Pod	✓ Running	14s	ready:1/1
└─ α rollouts-demo-774b46cfdc-5	AnalysisRun	✓ Successful	14s	✓ 1
└─└─ 6a10cc86-ffbc-4d2d-a71c-df3687acfea2.http-success-rate.1	Job	✓ Successful	14s	
# revision:4				
└─ [] rollouts-demo-754fbfc964	ReplicaSet	✓ Healthy	21m	stable
└─└─ [] rollouts-demo-754fbfc964-tjwb5	Pod	✓ Running	21m	ready:1/1
└─└─ [] rollouts-demo-754fbfc964-gh4z9	Pod	✓ Running	21m	ready:1/1
└─└─ [] rollouts-demo-754fbfc964-sbjfn	Pod	✓ Running	20m	ready:1/1

Démo 2 — Canary défaillant

Scénario

- ▶ Déploiement de bad-red
- ✗ **Réponses HTTP 500**
- ✗ **Sonde : 5/30 = 16 %**
- ✗ **Metric Failed → RolloutAborted**
- ✓ **Trafic gardé sur green, 0 downtime**

```
kubectl argo rollouts get rollout - rollback automatique
```

```

Name:          rollouts-demo
Namespace:     demo
Status:        ✗ Degraded
Message:       RolloutAborted: Rollout aborted update to revision 3: Metric "http-success-rate" assessed Failure
Strategy:      Canary
  Step:        0/9
  SetWeight:   0
  ActualWeight: 25
Images:        argoproj/rollouts-demo:bad-red (canary)
               argoproj/rollouts-demo:green (stable)

Replicas:
  Desired:     5
  Current:     6
  Updated:     1
  Ready:       4
  Available:   4

```

NAME	KIND	STATUS	AGE	INFO
rollouts-demo	Rollout	✗ Degraded	6m21s	
# revision:3				
rollouts-demo-7c84cfdcdb	ReplicaSet	✓ Healthy	58s	canary
rollouts-demo-7c84cfdcdb-qzt5x	Pod	✓ Running	58s	ready:1/
rollouts-demo-7c84cfdcdb-hdhqt	Pod	⦿ Terminating	17s	ready:1/
rollouts-demo-7c84cfdcdb-3	AnalysisRun	✗ Failed	17s	✗ 1
021d463e-6109-4b46-93de-600d302f6668.http-success-rate.1	Job	✗ Failed	17s	
# revision:2				

Bilan, avis & améliorations

Ce que j'ai appris

- ▶ Le progressive delivery en pratique
- ▶ L'imbrication GitOps ↔ Rollouts
- ▶ Les métriques comme contrat de qualité

Mon avis

- ✓ **Excellent compromis sécurité / vitesse**
- ▶ Pour des services critiques à fort trafic
- ✗ **Overkill pour un petit mono-service**

Améliorations possibles

- ▶ Trafic fin via Istio/NGINX · analyse Prometheus réelle (p95 + erreurs) · notifications Slack · Blue-Green avec tests preview · multi-cluster ApplicationSet

Merci de votre attention

Questions ?

```
kubectl argo rollouts get rollout rollouts-demo --watch
```

github.com/Louis-JB/argo-rollouts-demo